

Lab - 2:

Experiment No: 2

Dated:

AIM: Create a Blockchain using Python

Lab Objectives: To implement public and private Blockchain.

Lab Outcomes (LO): Demonstrate the concept of Blockchain in real-world Applications (LO4)

Task to be performed :

1. Make a copy of this [Google Colab Notebook](#)
2. Try to solve the errors in given Program
3. After successful execution of the Program in Colab Notebook.
 - **Add a method, create_Transactions**
 - Mine the block only when the transaction list is not null.
 - Remove the transactions from the list of transactions before mining.
 - **Modify the method, proof_of_work() to search for the golden nonce**
 - Cryptographic Puzzle is to have “000” leading zeros in the Block Hash
4. Download the code - [blockchain.py](#)
5. **Update the code to incorporate the changes in step 3 to the code in step 4.**
6. Follow the steps in [Manual](#) to demonstrate the working of Blockchain using Flask and Postman.

Tools & Libraries used :

- [Flask](#) : pip install Flask
- Download **Postman** from <https://www.postman.com/>
- Python Libraries : **datetime, jsonify, hashlib**

Instructions : (Prepare for viva for the following topics)

1. What is a Blockchain?
2. Process of Mining
3. How to check the validity of blocks in a Blockchain

Outcome :

1. Understood the concept of mining in Blockchain.
2. Implemented programs
 - Mine a Block
 - Display the chain in the Blockchain
 - Check whether the chain is valid
3. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.
4. Submit the hardcopy **by the 1st week of Sem**
(As per the instructions, submit a hard copy of the same).

Aim:

Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

1. Importing Required Libraries

- **datetime** → to timestamp each block
- **hashlib** → to generate SHA-256 hashes
- **json** → to encode blocks as JSON for hashing
- **Flask** → to run the blockchain API server

2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
- store the chain
- implement Proof of Work
- validate chain integrity

3. Flask Web API

Provides endpoints:

- `/mine_block` → mine a new block
- `/get_chain` → fetch full blockchain
- `/is_valid` → verify blockchain

Program And Output

🔗 Blockchain_Lab_2.ipynb

```
import datetime
```

```
import hashlib
```

```
import json
```

```
class Blockchain:
```

```
    def __init__(self):
```

```
        self.chain = []
```

```
        self.create_block(proof = 1, previous_hash = '0')
```

```
    def create_block(self, proof, previous_hash):
```

```

block = {'index': len(self.chain) + 1,

         'timestamp': str(datetime.datetime.now()),

         'proof': proof,

         'previous_hash': previous_hash}

self.chain.append(block)

return block


def get_previous_block(self):

    return self.chain[-1]


def proof_of_work(self, previous_proof):

    new_proof = 1

    check_proof = False

    while check_proof is False:

        hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()

        if hash_operation[:4] == '0000':

            check_proof = True

        else:

            new_proof += 1

    return new_proof

```

```

def hash(self, block):

    encoded_block = json.dumps(block, sort_keys =
True).encode()

    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):

    previous_block = chain[0]

    block_index = 1

    while block_index < len(chain):

        block = chain[block_index]

        if block['previous_hash'] !=
self.hash(previous_block):

            return False

        previous_proof = previous_block['proof']

        proof = block['proof']

        hash_operation = hashlib.sha256(str(proof**2 -
previous_proof**2).encode()).hexdigest()

        if hash_operation[:4] != '0000':

            return False

        previous_block = block

        block_index += 1

    return True

```

Creating a Blockchain

```

blockchain = Blockchain()

# Mining a new block

def mine_block():

    previous_block = blockchain.get_previous_block()

    previous_proof = previous_block['proof']

    proof = blockchain.proof_of_work(previous_proof)

    previous_hash = blockchain.hash(previous_block)

    block = blockchain.create_block(proof, previous_hash)

    response = {'message': 'Congratulations, you just mined a
block!',

                'index': block['index'],

                'timestamp': block['timestamp'],

                'proof': block['proof'],

                'previous_hash': block['previous_hash']}

    return response

# Getting the full Blockchain

def get_chain():

    response = {'chain': blockchain.chain,

                'length': len(blockchain.chain)}

    return response

# Checking if the Blockchain is valid

def is_valid():

```

```

        is_valid = blockchain.is_chain_valid(blockchain.chain)

        if is_valid:

            response = {'message': 'All good. The Blockchain is
valid.'}

        else:

            response = {'message': 'Houston, we have a problem. The
Blockchain is not valid.'}

        return response

print("Functions Menu:")

print("=====")

print("1. Mine a block")

print("2. Display the chain")

print("3. Check the validity of the chain")

choice = int(input("Enter your choice :"))

if choice == 1:

    print(mine_block())

elif choice == 2:

    print(get_chain())

elif choice == 3:

    print(is_valid())

else :

    print("Invalid choice")

```

```
▶ blockchain.transactions.clear()
```

```
▶ print(mine_block())
```

```
... {'message': 'No transactions available. Block cannot be mined.'}
```

```
.. Blockchain created successfully.  
   Number of blocks: 1  
   Pending transactions: []
```

```
... Transaction created:  
   {'sender': 'Soham', 'receiver': 'Som', 'amount': 100}  
   Pending transactions:  
   [{'sender': 'Soham', 'receiver': 'Som', 'amount': 100}]
```

```
{'message': 'Congratulations, you just mined a block!', 'index': 2,  
'timestamp': '2026-08-29 12:45:11.663353', 'proof': 533, 'previous_hash':  
'4f90c8de61702a9244f1ef476d4d534252914a71f6680590b51719b039ab2cb5',  
'transactions': [{'sender': 'Soham', 'receiver': 'Som', 'amount': 100}]}
```

```
▶ print("Pending transactions:")  
  print(blockchain.transactions)
```

```
... Pending transactions:  
   []
```

Golden Nonce: 533

Golden Hash:

0000c00870f23a23ae80377298491b091db400d575be0efbde5b310f2f763ed1

```
{'message': 'Congratulations, you just mined a block!', 'index': 2,
```



```
'timestamp': '2026-08-29 12:56:51.513576', 'proof': 533, 'previous_hash':  
'868b7c9382a462f696ec05f94b99ceb61829b23a34a883ec6b6e79b3285496cb',  
'transactions': [{ 'sender': 'Soham', 'receiver': 'Som', 'amount': 100}]}
```

Functions Menu:

=====

1. Mine a block
2. Display the chain
3. Check the validity of the chain

Enter your choice :1

```
{ 'message': 'Congratulations, you just mined a block!', 'index': 3,  
'timestamp': '2026-08-29 12:17:05.774682', 'proof': 45293,  
'previous_hash':  
'6fdf3aa52123e8447588edc40284caa76707b11f0d332a196755ec181788c804' }
```

Functions Menu:

=====

1. Mine a block
2. Display the chain
3. Check the validity of the chain

Enter your choice :2

```
{ 'chain': [{ 'index': 1, 'timestamp': '2026-08-29 12:15:38.932016',  
'proof': 1, 'previous_hash': '0'}, { 'index': 2, 'timestamp': '2026-08-29  
12:15:50.210942', 'proof': 533, 'previous_hash':  
'49d213822749edc6bb9cf6d3a1dee730d47e210166b2edc7f8eb97d25f485d94'},  
{ 'index': 3, 'timestamp': '2026-08-29 12:17:05.774682', 'proof': 45293,  
'previous_hash':  
'6fdf3aa52123e8447588edc40284caa76707b11f0d332a196755ec181788c804' } ] },
```

```
'length': 3}
```

Functions Menu:

=====

1. Mine a block
2. Display the chain
3. Check the validity of the chain

Enter your choice :3

```
{'message': 'All good. The Blockchain is valid.'}
```

Flask Web Api

```
app.py  X
app.py > mine_block
1  from flask import Flask, jsonify
2  from blockchain import Blockchain
3
4  app = Flask(__name__)
5
6  blockchain = Blockchain()
7
8
9  @app.route('/mine_block', methods=['GET'])
10 def mine_block():
11
12     if len(blockchain.transactions) == 0:
13         return jsonify({
14             'message': 'No transactions available. Block cannot be mined.'
15         }), 400
16
17     transactions = blockchain.transactions.copy()
18     blockchain.transactions.clear()
19
20     previous_block = blockchain.get_previous_block()
21     previous_proof = previous_block['proof']
22
23     proof = blockchain.proof_of_work(previous_proof)
24
25     previous_hash = blockchain.hash(previous_block)
26
27     block = blockchain.create_block(
28         proof,
29         previous_hash,
30         transactions
31     )
32
```

```

33     response = {
34         'message': 'Congratulations, you just mined a block!',
35         'index': block['index'],
36         'timestamp': block['timestamp'],
37         'proof': block['proof'],
38         'previous_hash': block['previous_hash'],
39         'transactions': block['transactions']
40     }
41
42     return jsonify(response), 200
43
44     (function) route: Any
45 @app.route('/get_chain', methods=['GET'])
46 def get_chain():
47
48     response = {
49         'chain': blockchain.chain,
50         'length': len(blockchain.chain)
51     }
52
53     return jsonify(response), 200
54
55
56 @app.route('/is_valid', methods=['GET'])
57 def is_valid():
58
59     valid = blockchain.is_chain_valid(blockchain.chain)
60
61     if valid:
62         response = {
63             'message': 'All good. The Blockchain is valid.'
64         }

```

```

65     else:
66         response = {
67             'message': 'Houston, we have a problem. The Blockchain is not valid.'
68         }
69
70     return jsonify(response), 200
71
72
73 @app.route('/add_transaction', methods=['GET'])
74 def add_transaction():
75
76     transaction = blockchain.create_Transactions(
77         "Soham",
78         "Som",
79         100
80     )
81
82     return jsonify({
83         'message': 'Transaction created',
84         'transaction': transaction
85     }), 200
86
87
88 if __name__ == '__main__':
89     app.run(debug=True)

```

```

D:\BLOCKCHAIN EXP2>dir
Volume in drive D is New Volume
Volume Serial Number is FA91-F62F

Directory of D:\BLOCKCHAIN EXP2

29-08-2026  18:47      <DIR>
29-08-2026  18:41                2,319 blockchain.py
               1 File(s)                2,319 bytes
               1 Dir(s)  312,453,849,088 bytes free

D:\BLOCKCHAIN EXP2>python
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 6
4 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from blockchain import Blockchain
>>> blockchain = Blockchain()
>>> print(len(blockchain.chain))
1
>>> exit

D:\BLOCKCHAIN EXP2>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production depl
oyment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 914-331-230
127.0.0.1 - - [30/Aug/2026 19:37:00] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Aug/2026 19:37:00] "GET /favicon.ico HTTP/1.1" 404 -

```

Soham Chainani's Workspace

Overview GET http://127.0.0.1:5000/ac No environment

GET http://127.0.0.1:5000/add_transaction Save Share

GET http://127.0.0.1:5000/add_transaction Send

Docs Params Authorization Headers 6 Body Scripts Settings Cook

Query Params

Key	Value	Descript...	Bulk Edit
Key	Value	Description	

Body 200 OK 24 ms 293 B

{ } JSON Preview Visualize

```

1 {
2   "message": "Transaction created",
3   "transaction": {
4     "amount": 100,
5     "receiver": "Som",
6     "sender": "Soham"
7   }
8 }
9

```

> ENVIRONMENTS
> DOCUMENTS
> SPECS

Soham Chainani's Workspace

Overview GET http://127.0.0.1:5000/mine_block

http://127.0.0.1:5000/mine_block

GET http://127.0.0.1:5000/mine_block

Send

Query Params

Key	Value	Description
Key	Value	Description

Body

200 OK - 20 ms - 495 B

JSON

```
1 {
2   "index": 2,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash":
5     "ea2e21df9e83446965e62850b7afb472d6f6e847a22ffd589ce28e
6     c7e48c5279",
7   "proof": 533,
8   "timestamp": "2026-08-30 20:11:25.554077",
9   "transactions": [
10     {
11       "amount": 100,
12       "receiver": "Som",
13       "sender": "Soham"
14     }
15   ]
16 }
```

Workspace

Overview GET http://127.0.0.1:5000/get_chain

http://127.0.0.1:5000/get_chain

GET http://127.0.0.1:5000/get_chain

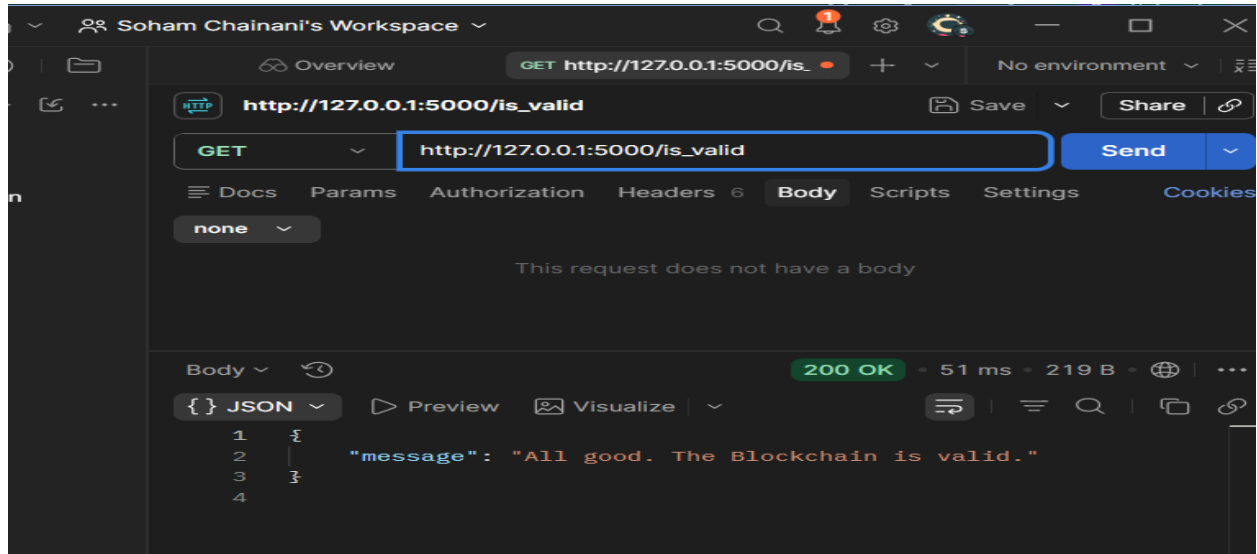
Query Params

Key	Value	Description
Key	Value	Description

Body

JSON

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-08-30 19:36:23.443802",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "ea2e21df9e83446965e62850b7afb472d6f6e847a22ffd589ce28ec7e48c5279",
13      "proof": 533,
14      "timestamp": "2026-08-30 20:11:25.554077",
15      "transactions": [
16        {
17          "amount": 100,
18          "receiver": "Som",
19          "sender": "Soham"
20        }
21      ]
22    }
23  ],
24  "length": 2
25 }
```



Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.